

orkflow::planning breakdown to ~workflow::ready-for-development and to maintain the WIP limits. Everyone is welcome to improve issues to make the ready for development.

Special labels

- the ~environments::parked label is used to signal that we don't intend to focus on an issue in the next 9-12 months

Milestone Board

The issues scheduled for a milestone can be tracked at Milestone Board.

This board contains all the necessary columns to track the workflow of the team, in particular:

- Labels of interest as outlined above
- One or more Milestone columns containing the planned work for the given milestone.

All the columns are prioritised top to bottom.

Once a team memeber self-assigns an issue on the Milestone Board, issue labels should follow the Engineering Workflow.

For Merge Requests, it's up to the author and the project they are contributing to, to decide if they want to use these ~workflow:: labels. It is not required to use them or keep them synced up with the Issue labels.

Planning

Team Domain Limitations

The Environments team size is currently too smal to fully support the entire scope of our feature categories, to signal our priorities and do meaningful work we maintain a list of Feature Categories where we only do Critical Maintenance:

- Auto DevOps
- Feature Flags
- Continuous Delivery
- Infrastructure as Code
- Release Orchestration

With Critical Maintenance we mean that we will be able to take on only p2/s2 and above Security, Scalability and Availability issues and only p1/s1 bugs otherwise classified, or issues considered impactful to fix by the Product Manager.

Issues falling out of the mentioned types will be marked with ~environments::parked label and we will ignore their SLO.

Issue Weighting

The weights we use are:

Weight	Extra investigation	Surprises	Collaboration
-----	-----	-----	-----
1: Trivial	not expected	not expected	not required
2: Small	possible	possible	possible
3: Medium	likely	likely	likely
5: Large	guaranteed	guaranteed	guaranteed

The above table is contextual. For example, domain knowledge, experience levels, and time at GitLab can impact an engineer's perspective on whether an issue requires Extra Investigation or Surprises.

Weights are not set in stone. We do our best to get it right during refinement, but we want to be transparent and accurate. If an issue is taking more effort than is reflected in the existing weight, the DRI on the issue is encouraged to change the weight. We want accurate documentation of the level of effort that was required.

By giving a weight 1 to an issue, we're saying "we can't benefit from this issue being broken down into smaller units of shippable work."

Anything 5 or larger should be broken down, these should not be ready for development. We would likely turn a 5 into an epic, into a research and implementation issue or a technical discovery.

Occasionally, a proof-of-concept (POC) is necessary to determine a feasible technical path. When one is required, the engineer will create a POC issue that contains the context of the research to be conducted along with the goals of the POC. This issue will be scheduled for work before any further breakdown of tasks is performed. Once the technical path is clear, the engineer can proceed to weight the issue and/or break down the issue further to guide implementation. Every POC issues should contain a list of questions we want to answer, the definition of done should include the answers and suggested next steps.

Not all POCs will be successful, and that is OK! Some avenues of research may not be successful, and the POC will have saved us from investing significant time in a solution that will not meet our needs. The goal is early feedback and fast iteration.

As a note, designers use the design weight labels instead of using the weight input within the issue, which is reserved for engineering.

Weight, Velocity, and Planning

We intentionally leave the term "velocity" undefined and do not use it in planning workload capacity for the team.

We leave the question of interpreting summed weights open to each unique situation.

When making decisions about how much work the team can take on for a milestone, we trust individual impressions and instincts reflected in the discussions that take place in the planning issue and the refinement process. The weighting system helps foster these discussions.

The GitLab Terraform Provider is managed by the Environments group.

Feature development

Our goal is to move towards a continuous delivery model so the team completes tasks regularly, and keeps working off of a prioritized backlog of issues. We default to team members self-scheduling their work:

- Team members self-assign issues from the Milestone Board that are in the workflow:ready for development column and has the current milestone.
- ~Deliverable issues take priority over any other work, as they are the main focus of each milestone and inform our say-do ratio.
- Once a team member has completed their assigned issues, they are expected to go to the Milestone Board and assign themselves to the next unassigned issue from the current milestone.
- If there are no more issues in the current milestone, engineers are expected to assign themselves to the next unassigned workflow:ready for development issue.
- The issues on the board are in priority order based on importance (the higher they are on the list, the higher the priority). This order is set by the product manager.
- If all issues are assigned for the milestone, team members are expected to identify the next available issue to work on based on the team's work prioritization (see below).
- While backstage work is important, in the absence of specific prioritization, the team will have a bias towards working on bug or feature categorized issues.

~Environments::EngineeringChoice process

While diligently pursuing our objectives, we also recognize the significance of work that resonates personally with our engineers. To facilitate this, we have introduced the "~Environments::EngineeringChoice" label. Here's how it works:

1. Selection During Milestones: In each Milestone Plan, engineers are encouraged to select up to five issues (total for the group) that they find particularly interesting or valuable, marking them with the "~Environments::EngineeringChoice" label. These issues should improve GitLab product or developer experience but they don't have to be in the ~"group::environments" domain.
1. Limit per Milestone: To maintain focus, no more than five issues should be labeled with "~Environments::EngineeringChoice" within a single milestone.
1. Priority After Deliverables: Once all mandatory ~Deliverables are completed, the next priority is to address issues labeled "~Environments::EngineeringChoice."
1. Refined issues only: Before applying the "~Environments::EngineeringChoice" label, the issue should be workflow:ready for development and accordingly needs a weight.
1. Maximum issue size: To rule out likely surprises and extra investigation, only issues with weight 1-2 are acceptable for "~Environments::EngineeringChoice".
1. Tracking in Milestone Planning: Progress and choices under the "~Environments::EngineeringChoice" category will be monitored and recorded in a specific section of the Milestone Planning issue.

Bug fixing and prioritized work

In every milestone plan, we compile a list of bugs due in the coming milestone based on the severity SLA.

When severity labels are assigned/changed on a ~type::bug issue, we aim to set/adjust the issue due date at the same time.

everyone is encouraged to set the deadline based on the date of the last severity label update and the SLA for the given severity.

Best practices for managing bug issues

Goals:

- Effectively track and label bug related issues.
- Ensure bug Due Dates are not missed due to a lack of DRIs on sub issues.
- Ensure the team is aware that help is needed in a specific area on a bug that already has an overall DRI.

Context:

- Single part bug issues
 - Some bugs only require a single cohesive effort to resolve. For example an isolated backend fix that requires no database or frontend changes. In these cases, the DRI of the bug issue is the person doing the work, and all work is tracked in the bug report issue.
- Multi-part bug issues
 - In other cases, a bug issue may result in work across frontend, backend, and database. This can result multiple engineers working separately as DRIs of individual issues that all contributing to solving the bug. Multiple issues are needed.

Problem:

- Without a clear structure of issues for multi-part bugs, it's difficult for the team to know how to help and how to plan. This difficulty can negatively impact our say-do ratio.

Best practices for managing multi-part bug issues:

- The original bug issue should be promoted to an epic.
- The original DRI becomes the overall bug epic DRI (note this on the epic).
- New sub issues representing each part of the work should be created on the epic.
- The new issues should be noted as blocking the epic.
- Except for severity and priority, Labels should be copied over.
- Due dates should keep in mind the due date of the epic, which is based on severity and priority.
- Deliverable labels should be applied if the epic is deliverable.
- DRI can use the Milestone Planning issue and/or reach out to relevant team members to ask if there's availability within the Due Date. cc your engineering manager so they can give a high level thumbs up/thumbs down regarding the change in priority.

Bug resolution process

The entire bug resolution process includes the following phases in order:

1. GitLab Issue triage procedure: we have a handbook section we can follow [here](#)
2. Environment's team refinement process
3. Planning
4. Reprioritization. The EM will change unplanned p3 bugs that have had no activity to p4 and remove the due date.

Putting the process together

- Bug must be refined a milestone before it is due. This is done by the refinement DRI.
- Bug fix must be planned for a milestone that ends before the bug's due date. This takes place on the milestone planning issue.
- Reprioritization will have a dedicated section in the milestone planning issue.
- Outdated bugs are closed in accordance with the existing handbook practice

Best practices

- Read the issue triage handbook page
- Ask the reporter for detailed steps to reproduce the problem, including minimal setup and expected versus actual outputs.
- Request relevant documentation to validate the unexpected behavior, or an explanation if no documentation exists.
- If the reporter is a GitLab team member, inquire if there are any insights on the impact of the issue, such as the number of users affected or specific features involved, to help prioritize the resolution.
- Partner with the PM if you think it may not actually be a bug.

Say-do ratio

Our team keeps track of their commitment with say-do ratios, two metrics are important: say-do and reprioritized say-do

- Say-do only applies to ~Deliverable issues.
- By the 17th of the month the ~Deliverable label is applied to the upcoming milestone issues by the EM.
- We aim roughly to assign one ~Deliverable for each engineer, this may change milestone by milestone.
- Any issue that has the ~Deliverable label at that point is considered as promised to be delivered and is part of our say-do ratio.
- If at any time during the milestone a ~Deliverable label is removed or the issue is removed from the milestone that issue does not count anymore in the reprioritized say-do metric, but

still does count for say-do.

We aim to achieve 100% re-prioritized say-do and at least 80% say-do.

Example

- In the milestone 15.11 we have 10 ~Deliverable issues labeled as such by the 17th of March 2023
- Along the way we realise that 5 of those ~Deliverable issues will not make it, and reasonably before the end of the milestone, we move them to 16.0
- At the end of the milestone there is an hiccup and of those 5 remaining issues, 1 is not completed.

Our say-do ratio would be 40% (4 out of 10)

Our reprioritized say-do would be 80% (4 out of 5)

MR reviews

Team members should use their best judgment to determine whether to assign the first review of an MR based on the DangerBot's suggestion or to someone else on the team. Some factors in making this decision may be:

- If there is known domain expert for the area of code, prefer assigning the initial review to them.
- Does the MR require a lot of context to understand what it is doing? Will it take a reviewer outside the team a long time to ramp up on that context?
- Does the MR require a lot of dev environments setup that team members outside the Environments group are likely to not have?
- Is the MR part of a larger effort for which a specific team member already has all the context?

As team members and domain experts, both the MR author and initial reviewer are encouraged to share the broader context before, during, and throughout the review process to assist maintainers in conducting efficient reviews. This context may cover:

- Known limitations;
- Edge cases;
- Implementation reasoning;
- Links to relevant references.

Providing context helps streamline the review process and invites a broader pool of maintainers to our domain (example).

Handling Deferred UX

Team members should make their best effort to resolve UX issues as they come up during MR reviews. However, there are times where the changes requested or feedback given would significantly slow down velocity. For the sake of efficiency and iteration, a Deferred UX issue must be opened to follow up on the feedback.

In these instances, the engineer who authored the original MR should assign themselves the issue and become the DRI to evaluate the UX feedback. This may mean reaching out to the team's Product Designer to ensure the feedback is actionable and resolving the debt is prioritized appropriately during the following milestone planning. For example, for Deferred UX issues opened in the 16.3 milestone, engineers should evaluate and ensure appropriate prioritization of the issue during the planning of the 16.4 milestone. This does not mean that the issue must be resolved during the 16.4 milestone, but that the issue is placed into the appropriate step of our product development flow, or closed if appropriate.

This helps to ensure that Deferred UX issues are resolved in a timely manner, keeping with the overall goals of the group and adherence to broader engineering workflows.

Epic Ownership

The Environments group uses epics to describe features or capabilities that will increase the maturity of the Environments categories over time.

Each epic should be owned by an engineer who is responsible for all technical aspects of that epic. The engineering DRI will work closely with the Product Manager and Product Designer to understand the requirements and create issues that encapsulate the technical work required during the design/solution validation phases and build track of the Product Development Flow. Each issue needs to be weighted and contain enough information in the description area for any other engineer on the team to be able to pick up that work.

For the duration of building the epic, the engineer does not need to be the only person implementing the issues. They should keep watch of the work that is done on the issues so that they can verify that the work is progressing correctly. If there are problems with the work, or lengthy delays, they need to make sure the Product Manager and Engineering Manager are aware.

When work is nearing completion, the engineer should make sure that any additional issues that may have come up during the build process are either addressed, or scheduled for work. Additional issues should be created and added to the epic. This will help to make sure that we do not build up technical debt while building.

Finally, they should also monitor any work that needs to occur while rolling out the Epic in production. If there are rake tasks, database migrations, or other tasks that need to be run, they need to see those through to being run on the production systems with the help of the Site Reliability counterpart.

This places a lot of responsibility with the DRI, but the PM and EM are always there to support them. This ownership removes bottlenecks and situations where only the PM or EM is able to advance an idea. In addition, the best people to decide on how to implement an issue are often the people who will actually perform the work.

To declare an ownership, insert DRI: <your-gitlab-handle> at the top of the epic description.

Example.

Quality Processes

Maintaining a high standard of quality is a critical factor to delivering winning products.

Within the Environments group we use the following processes and best practices to ensure high quality.

1. We ensure each MR is accompanied with meaningful unit tests and integration tests.
1. For each major feature we develop and maintain End to End tests that run nightly and confirm no regressions have been introduced to critical paths.
1. On a weekly basis, we review our Triage report for bugs and regressions and take the appropriate action.
1. We review the quality dashboard each milestone to track our long term progress at improving quality.

End to End Testing

The Environments group uses GitLab QA for End-to-End testing. We have guidelines for how our team is leveraging these tests.

gitlab-agent QA bot

In feedalertsconfigure we have a bot that runs tests at this project

If this bot alerts of a failed pipeline, we should treat these the same as a broken master branch.

- Check the pipeline for intermittent errors (and retry if this is the case)
- Otherwise create an investigation issue to dig further/fix.

Error Budget

Our target availability is 99.9%

Error Budget failure DRI process

Each week we receive an Error Budget report in cd-section on Slack if we are under our target availability.

An engineer might be assigned as a DRI to look into this.

The DRI is neither expected to determine a root cause nor propose a solution on their own.

The DRI should instead reach out to the Observability team for support.

Async Issue Updates

In order to optimize async collaboration across a big team we use issue updates to share

progress completed on a specific issue or epic.

Weekly updates on progress and status will be added to each issue by its assignee. A weekly update may be skipped if there was no progress. It's preferable to update the issue rather than the related merge requests, as those do not provide a view of the overall progress. This applies to issues with the labels `workflow::in dev` or `workflow::in review`

The status comment should include what percentage complete the work is, the confidence of the person that their estimate is correct and, a brief note on what was done. It's perfectly acceptable to have multiple updates if more than one DRI is working on the issue.

As a part of the async update it's important to verify that the issue and related MRs workflow labels are correctly set.

Example

markdown

Async status update

- Complete: 80%
- Confidence: 90%
- Notes: expecting to go into review tomorrow

To simplify the work of adding and keeping track of async updates TalTal can be used.

Career Development and Promotions

We want every team member to be advancing in their Career Development.

We follow the Engineering Department Career Development Framework.

Maximize asynchronous performance in this team

Async practices are particularly important to us because we live in time zones that do not afford much, if any, overlap during our working hours.

To maximize our asynchronous performance, we should follow the GitLab Communication guideline. More specifically, the following points are important:

- Have an SSOT discussion page (Issue or MR). This is the main collaboration point that everyone can get the latest information quickly.

The description section should contain essential and up-to-date information, such as:

- What's the problem to solve?
- Who's the DRI in charge of making the decision?
- What's the acceptance criteria (e.g. user experience goal)?
- Is anything out of scope?
- What proposals do we have?
- What are the PROs/CONs and technical difficulty of each proposal?
- Whose approval do you need for making the decision?
- When is the due date to make the decision?

- FAQ
- The DRI keeps the description updated with latest information based on any decisions made in threads.
- When a team member is asked to give input, they should respond as soon as possible to unblock discussions.
It's also fine to respond that you don't have any feedback or can't take time for it, so that the DRI can avoid waiting for your response.
- If the DRI didn't get much progress from the asynchronous communication, the DRI should schedule a synchronous meeting or reach out to broader audiences.
- When the DRI schedules a sync meeting, they should make sure that agendas are prepared before the meeting starts.

Monthly Showcases DRI

We participate in the OPS showcase initiative, to facilitate the selection of topics, the creation of the issues and content we have a Showcase DRI which will:

- Ensure every month at least a showcase issue is created and linked in the right issues/epic
- Facilitate the selection of the topic of each showcase, paying attention to give space to everyone in the team
- Help whoever is creating the content with video creation and issue description
- Ultimately is the showcase DRI responsibility that a showcase issue is produced and ready in time

Currently the showcase DRI for FY24Q3 is: @annavovchenko

How to work with us

Default to GitLab Issues

Why

We think that using GitLab Issues as much as possible is the best way to align with our values of Transparency and Efficiency. Using Issues gives us the greatest chance of collaboration, reusing any work done, and documenting the request and outcome in a findable, persisted way.

How

Follow the guidance in our request for help documentation.

How to contribute to Auto DevOps

Read our specific GDK instructions as well as our handbook entry on what existing testing does and how to develop features for Auto DevOps.

Useful links for contributing to Auto DevOps

- Tips and Troubleshooting
- Useful Commands

- How to work with slow connections
- Enabling premium features for development purposes
- Grafana query for complete Auto DevOps pipelines

Shared Cloud Infrastructure

The Environments group has access to a shared GCP project which can be used for demos, experiments, or to host auxilliary services.

The project id is deploy-stage-shared-i-e55e01cb and was created and provisioned using the following ARs:

- <https://gitlab.com/gitlab-com/it/infra/issue-tracker/-/issues/493> (internal)
- <https://gitlab.com/gitlab-com/it/infra/issue-tracker/-/issues/495> (internal)

If you need to create permanent infrastructure in that GCP project, it's encouraged to do it with Terraform

to easily share and document the setup with the entire group. You can use this GitLab group to host the project.

If the infrastructure is temporary, you can manage it with whichever tools you prefer.

Example/Demonstration projects

When you need to create an example project for demonstration, consider having it in the example group instead of your personal namespace.

This allows us to collect all of the knowledge under the same place. Also, this example group has EEP license by default.

SECTION: engineering/devops/ops/deploy/environments/autodevops.md

Current end-to-end tests

At the time of writing this entry there are two end-to-end test files:

- autodevopstemplatesspec.rb
- createprojectwithautodevopsspec.rb

under the qa/specs/features/browserui/7configure folder of the qa directory. There are not presently any end-to-end api-based tests.

Where the tests execute

Non-quarantined test execution environments

The Configure AutoDevOps Templates template: express|rails|spring works with Auto DevOps test is a tagged to run only in the :staging environment and therefore runs in :

- staging
 - qa-triggers-browserui-7configure

Quarantined tests

The following tests, if they are tagged quarantine, will run in these environments:

Configure AutoDevOps Templates template: express|rails|spring works with Auto DevOps

- staging
 - qa-triggers-browserui-quarantine

Configure your GDK to run the tests

Follow the documentation on how to Use GDK with a GKE cluster.

NOTE: The default instructions below prescribe using "the GCP project called gitlab-internal-153318 for development and testing," which, in order to create a cluster on said project, requires a set of permissions your humble author did not possess, whereas I have found that using the project gitlab-qa-resources was more compliant with my permission set.

Run the tests

To run the createprojectwithautodevopsspec tests, ensure you have GDK set up to run a GKE cluster (see above instructions) and call the test from the `../gitlab/qa/` directory.

Example: `CHROMEHEADLESS=true bundle exec bin/qa Test::Instance::All https://gdk.test:3000 --qa/specs/features/browserui/7configure/autodevops/createprojectwithautodevopsspec.rb --tag orchestrated --tag kubernetes`

or

`gitlab-qa Test::Instance::Image EE --qa/specs/features/browserui/7configure/autodevops/createprojectwithautodevopsspec.rb --tag orchestrated --tag kubernetes` to run orchestrated.

To run the autodevopstemplatesspec.rb tests, it is easiest to execute the tests against the staging environment by following these instructions for setting up your environment variables.

NOTE: You will need access to the "QA Vault" in 1password. If you require access, you can fill out an individual access request from [here](#)

Example: `gitlab-qa Test::Instance::Staging --qa/specs/features/browserui/7configure/autodevops/autodevopstemplatesspec.rb` to run orchestrated (append `--tag quarantine` to run quarantined tests)

Useful links for contributing to Auto DevOps

- Tips and Troubleshooting
- Useful Commands
- How to work with slow connections

- Enabling premium features for development purposes
- Grafana query for complete Auto DevOps pipelines

 For information as to why we're running master pipelines on qa-mirror see the following issues:

- <https://gitlab.com/gitlab-org/gitlab-qa/-/issues/423>
- <https://gitlab.com/gitlab-org/omnibus-gitlab/-/issues/4717>

SECTION: engineering/devops/ops/deploy/environments/quality.md

Overview

The goal of this page is to document how the Environments group uses the GitLab QA framework (video walkthrough) to implement and run end-to-end tests.

```
<figure class="videocontainer">  
  <iframe src="https://www.youtube.com/embed/enfx6tiz5WY" frameborder="0"  
allowfullscreen="true"> </iframe>  
</figure>
```

Supporting slides for the above walkthrough

Why do we have them

> End-to-end testing is a strategy used to check whether your application works as expected across the entire software stack and architecture, including the integration of all micro-services and components that are supposed to work together.

The testing strategy for this level of the pyramid can be found under the Environments Test Plan epic.

Where are they

In the GitLab repository, the End-to-End tests for the Environments group are located at:

- qa/qa/specs/features/api/7configure/
- qa/qa/specs/features/browserui/7configure/
- qa/qa/specs/features/ee/api/7configure/
- qa/qa/specs/features/ee/browserui/7configure/

Limitations

End-to-End tests for Environments are running only in staging and staging-canary environments. It is not possible to run these kind of tests against your local GDK without directly tweaking the test code.

FAQ

I triggered the e2e:test-on-omnibus job. Where do I find the tests?

If you have an MR and want to make sure it is running the End-to-End tests, please trigger the manual e2e:test-on-omnibus job on the pipeline of your MR. After the pipeline runs there will be a note on your MR titled "Allure report" with a test report link.

It is also possible to see which jobs failed in the e2e:test-on-omnibus pipeline directly.

Note: Currently, the tests most relevant for Environments run only on staging and staging canary environments.

How can I find tests results for staging or staging canary?

Tests running in staging need access to <https://ops.gitlab.net/gitlab-org/quality/staging/>.

The results of test runs can be found also in the e2e-run-staging Slack channel. The jobs in the staging pipeline that are relevant to Environments are:

- ee-qa-api
- qa-browserui-7configure

Which resources tests use and where can they be found?

Environments End-to-end tests use the gitlab-qa-resources project in GCP to spin up GKE clusters.

What's the GitLab Agent for Kubernetes client version in use?

The version in use can be found on the Runtime Environment Variables file.

Troubleshooting

Please reach out in the sdeveloperexperience channel.

Helpful Documentation

- Testing Guide - End-to-End Testing
- GitLab QA orchestration tool
- Run QA tests against your GDK setup
- Beginner's Guide to writing End-to-End tests

SECTION: engineering/devops/ops/package/_index.md

Mission and Vision

The Package stage creates a secure environment where both source code and dependencies can live by allowing you to publish, consume, and discover packages across a variety of languages and platforms all in one place. Our vision is to be our customers' single source of truth for storing and distributing packages and container images across the entire DevOps lifecycle.

Team Structure

The Package stage consists of two groups:

1. Package:Container Registry - Backend Go service for OCI-compliant container image storage
2. Package:Package Registry - Package management features within GitLab Rails application

Team Members

The following teams make up the Package stage:

```
{{< team-by-manager-slug manager="crystalpoole" team="Container Registry" >}}
{{< team-by-manager-slug manager="crystalpoole" team="Package Registry" >}}
```

Stable Counterparts

```
{{< engineering/stable-counterparts manager="crystalpoole" role="[,&] Package" >}}
```

Package Format DRIs

Format	DRI
npm	@dmeschcharakou
Maven	@10io
PyPI	@radbatnag
NuGet	@mkhalifa3
Terraform	@radbatnag
Generic	@dmeschcharakou

How We Work

Milestone Planning Process

Our milestone planning aligns closely with our product roadmap and engineering commitments. The process balances delivery of strategic initiatives with operational needs like security issues and customer bugs.

Product Manager responsibilities:

- Creates milestone planning issue with clear goals based on roadmap
- Defines key projects and expected outcomes
- Ensures security issues and customer bugs are considered
- Maintains alignment with product strategy and roadmap

Engineering Manager and DRI responsibilities:

- Reviews carryover work from previous milestone
- Selects next set of issues from their projects to meet roadmap timelines
- Evaluates realistic delivery capacity
- Surfaces risks and timeline impacts early

- Updates roadmap dates if needed

Issues are prioritized as:

- Package:P1: Committed work for the milestone
- Package:P2 + Stretch: Stretch goals if capacity allows

Non-roadmap items that may be planned include:

- Security issues (to meet SLAs)
- Customer bugs
- Minor reliability/performance fixes

Instrumentation is a key requirement for all features to measure impact and effectiveness. This helps us validate adoption and make data-driven decisions.

Issue Management

Async Issue Updates

The purpose of async updates is to communicate progress and allow others to prepare for upcoming work as necessary. In an all-remote culture, we keep the updates asynchronous and put them directly in the issues.

The async update communicates the progress and confidence using an issue comment and the milestone health status. Add a comment in your issue with the title Async Update once per week, or when something notable happens with regard to the issue. It's preferable to update the issue rather than the related merge requests.

The async update comment should include:

- what percentage complete the work is, in other words, how much work is done to put all the required MRs in review
- the confidence of the person that their estimate is correct
- notes on what was done and/or if review has started
- it could be good to include whether this is a front end or back end update if there are multiple people working on it

Issue Weighting Guidelines

Weight	Description	Confidence Level
1	Trivial	Well understood, no investigation needed, exact solution known
2	Small	Well understood, minimal investigation needed, few surprises expected
3	Medium	Well understood but requires investigation, some surprises expected
Larger	Should be broken down into smaller issues	

An issue with weight 1 should take no more than 2 days to complete.

Current Projects

Container Registry

- Tag immutability (DRI: @jdrpereira)
- Protected tags (DRI: @rchanila)
- Metadata database GA (DRI: @hswimelar)
- Database load balancing (DRI: @jdrpereira)
- Background migration support (DRI: @suleimiahmed)

Package Registry

- Virtual Registry support (Maven, npm, PyPI, NuGet) (DRI: @10io)
- Dependency Firewall (DRI: @dmeshcharakou)
- Package format improvements (DRI: @radbatnag)
- Test stability enhancements (DRI: @radbatnag)

Technical Guidelines

Breaking Changes Process

Breaking changes can be disruptive to our customers' workflow and we attempt to avoid them or reduce the impact. When a breaking change is unavoidable, we provide as much visibility and advance notice as possible.

1. Announce deprecations, breaking changes, and removals at least 3 milestones before implementation
2. Before major version milestone:
 - Implement breaking change with a feature flag
 - Feature flags are mandatory unless explicitly justified
3. In major version milestone:
 - Roll out the feature flag
 - If stable, open feature flag cleanup MR
 - If issues occur:
 - Fix before milestone end and re-roll out, or
 - Disable flag and defer to next major version

Releasing New Package Formats

We follow a structured process for releasing new package formats:

1. Experiment Phase
 - Feature flag enabled on staging
 - Comprehensive testing:
 - Small and large package operations
 - Performance with hundreds of packages
 - All supported authentication methods
 - Custom GitLab-specific features
 - Document any issues in feature flag rollout issue
2. Beta Phase
 - Limited GitLab.com rollout

- Dogfooding with internal teams
- Selected customer testing
- Bug triage and resolution

3. Generally Available

- Full documentation update
- Feature flag removal
- Self-managed release
- Release communications

Alert and CI Flake Management

The team monitors the Slack channel `gcontainer-registryalerts` for service alerts and CI notifications. Process for handling alerts:

1. Everyone is responsible for monitoring alerts during their working hours

2. When a new alert appears:

- Add 🟡 emoji to signal you're investigating
- Review alert details (runbook, dashboard, pipeline, Sentry issue)
- Use available resources to evaluate the problem
- Determine if safe to ignore based on:
 - Existing issue coverage
 - Auto-resolution
 - Logs showing resolution
- If not safe to ignore:
 - Review production and incidents-dotcom channels
 - Consider reporting an incident
 - Share details in `gcontainer-registry`
- Add a comment thread to document your review
- Add 🟢 emoji when resolved

Alert Occurrence Template

markdown

Alert Occurrence Update

- Occurrence Count: X (previously Y)
- Date/Time: [Insert timestamp]
- Last occurrences: [Insert slack link]

Key Resources

- Logs - Non-production
- Logs - Production
- Dashboard - Overview
- Profiler

Product Context

Container Registry History

Originally launched in milestone 8.8, the Container Registry integrated Docker Distribution registry into GitLab. A key challenge was storage management - deleted images weren't actually removed from storage without downtime-requiring garbage collection.

To address this and enable future features, we:

1. Forked Docker Distribution
2. Implemented online garbage collection
3. Added a metadata database
4. Created cleanup policies

This work enables future capabilities like:

- More robust API and UI
- Enterprise features (image signing, protection)
- Improved stability and reliability

The registry adheres to OCI standards for Image and Distribution specifications while maintaining backward compatibility with Docker specifications.

Measuring Results

Key Performance Indicators

- Lighthouse Metric: $(\text{Count of Packages Published} + \text{Count of Containers Published}) / \text{Billable Users}$
- Monthly Active Users (GMAU)
- Error Budget compliance
- Say-Do ratio for delivery commitments
- Operating costs:
 - Storage costs
 - Data transfer costs

Dashboards

- Container Registry
- Package Registry Metrics
- Error Budgets

Additional Resources

- Package Quality Guidelines
- Package Jobs To Be Done
- Container Registry Documentation
- Package Registry Documentation
- Team YouTube Channel (includes demos, team meetings, customer summaries, and user interviews)

SECTION: engineering/devops/ops/package/jtbd.md

Overview

The goal of this page is to create, share and iterate on the Jobs to be Done (JTBD) and their corresponding job statements for the Package stage. Our goal is to utilize the JTBD framework to better understand our buyers' and users' needs.

Goals

Utilize JTBD and job statements to:

- Understand our users' motivations.
- Validate identified use cases and solutions.
- Continuously test and iterate features to ensure we are meeting our customers' and users' needs.
- Create a transparent view for our stakeholders into the current and future state of the product.

Challenges

- Most customers do not purchase GitLab solely for the Package stage. So, there will be assumptions in the JTBD that from the buyer's perspective, the motivations for 'buying' Package are the same.

Jobs To Be Done

```
{{% product/jtbd "Package" %}}
```

```
## SECTION: engineering/devops/ops/package/quality.md
```

Testing Strategy

Local Development Testing

1. Unit Tests

- Container Registry (Go): make test
- Package Registry (Rails): bundle exec rspec ee/spec/models/packages/
- Frontend: yarn jest ee/spec/frontend/packages

2. Integration Tests

- Test against different storage backends
- Verify API endpoints
- Check authorization rules

End-to-End Testing

The Package group uses GitLab QA for E2E testing. This is crucial for features that integrate multiple services.

Common Test Scenarios

1. Container Registry

```
ruby
  Publishing a container image
  Resource::Container::Image.fabricateviaapi! do |image|
    image.name = 'my-image'
    image.repositoryname = 'my-project'
  end
```

2. Package Registry

```
ruby
  Publishing an npm package
  Resource::Package::Npm.fabricateviaapi! do |package|
    package.name = 'my-package'
    package.version = '1.0.0'
  end
```

Test Environment Setup

Local Testing with GDK

1. Set required environment variables:

```
bash
export QADEBUG=true
export WEBDRIVERHEADLESS=false
```

2. Ensure GDK is using a loopback interface

3. Map hostname to the loopback interface

Testing File Uploads

Test uploads against multiple storage backends:

1. Local storage (default)

2. Object storage:

- GCP (recommended for GitLab.com parity)
- MinIO (local development)
- AWS S3
- Azure

HackyStack Demo Environment

For ephemeral demos with Omnibus and Container Registry:

1. Login to <https://gitlabsandbox.cloud>
2. Select Cloud Account ID dev-package-container-96a3ff34
3. Get credentials from View Credentials tab
4. Launch environment using Play icon
5. Run CI/CD pipeline
6. Manually trigger Deploy action
7. Access demo project using provided credentials
8. Clean up by triggering Destroy job when done

Testing Tools

Seeding Utilities

1. Container Factory
 - Quick creation of test images
 - Supports multiple architectures
2. Container Registry Cloud Seeder
 - Bulk image creation
 - Load testing support

Troubleshooting

Common Issues

1. Storage backend connectivity

bash

Check GCS credentials

gsutil ls gs://your-bucket

Verify S3 access

aws s3 ls s3://your-bucket

2. Registry authentication

bash

Test registry login

docker login registry.gitlab.com

3. Package publishing failures
 - Verify project/group permissions
 - Check storage quotas
 - Validate package metadata

Documentation

- GitLab QA Framework

- End-to-End Testing Guide
- Container Registry Development
- Package Development

SECTION: engineering/devops/ops/project-plans/_index.md

Key projects

See Ops Key Projects for context.

- Service Desk Ticket Work Item
- CI Catalog
- Container Registry V2 Self-Managed
- Secrets Manager
- GCP Secrets Integration

SECTION: engineering/devops/ops/project-plans/cluster-web-terminal.md

Weekly Project Plan

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/11015>

Milestone 16.3 (June 18, 2023 - July 17, 2023)

Goals

- x] [Allow WebSockets KAS connections
- x] [Support WebSocket Connections from Browser

Milestone 16.4 (August 18, 2023 - September 17, 2023)

Goals

- -] [Design and craft a container that will allow for the shell environment to exist
- x] [Add Pod Attach API to the cluster integration javascript client

Week of Aug 28-Sept 1

- The bulk of the work is focused around updating the cluster javascript client, the DRI for this task had some PTO time
- The focus has shifted towards a p1/s1 security issue that then revealed to be less impactful and has been downgraded to p4/s4

Week of Sept 4-8

- [x] forecast: Updating the javascript client library to connect to the cluster is completed

or close to completion

- [-] forecast: The discussion around designing the container that will receive the connection proceeds and we have an initial tech proposal

Week of Sept 11-15

- [x] forecast: The discussion around designing the container that will receive the connection proceeds and we have an initial tech proposal

Milestone 16.5 (September 17, 2023 - October 10, 2023)

Goals

- x] [Design and craft a container that will allow for the shell environment to exist
- [-] Add the web terminal interface behind a feature flag

Week of Sept 15-22

- [x] forecast: The first version of the container is prepared and the tooling to automatically release it are clear and a plan to create them is ready

Week of Sept 25-29

- -] forecast: [<https://gitlab.com/gitlab-org/gitlab/-/issues/418264> and <https://gitlab.com/gitlab-org/gitlab/-/issues/418266> are completely refined
- -] forecast: [<https://gitlab.com/gitlab-org/gitlab/-/issues/418264> and <https://gitlab.com/gitlab-org/gitlab/-/issues/418266> are both picked up and a draft MR is ready

Week of Oct 02-06

- -] forecast: [<https://gitlab.com/gitlab-org/gitlab/-/issues/418264> and <https://gitlab.com/gitlab-org/gitlab/-/issues/418266> are completely refined
- -] forecast: [<https://gitlab.com/gitlab-org/gitlab/-/issues/418264> and <https://gitlab.com/gitlab-org/gitlab/-/issues/418266> are both picked up and a draft MR is ready
- x] We have shift our attention to create a [Design Document for the Cluster Web Terminal
- x] A demo [video with the current status of the feature has been created to further the discussion with the team

Week of Oct 09-13

- -] [The Design Document for the Cluster Web Terminal is finalised and merged.

Milestone 16.6 (October 17, 2023 - November 10, 2023)

Goals

- x] Finalise [the Design Document for the Cluster Web Terminal

Week of Oct 16-20

- -] forecast: [The Design Document for the Cluster Web Terminal is finalised and merged.

Week of Oct 23-27

- x] forecast: [The Design Document for the Cluster Web Terminal is finalised and merged.

Week of Oct 30 Nov 03

- We hit the goal for this milestone a bit earlier, this allows us to recover some pace on the project.
- x] forecast: now that [The Design Document is merged the necessary issues are created and refined

Week of Nov 06-10

- [-] forecast The issues created as the followup to the design document are refined:
 - -] [<https://gitlab.com/gitlab-org/gitlab/-/issues/426285>
 - -] [<https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/issues/479>
 - -] [<https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/issues/480>

Milestone 16.7 (November 13, 2023 - December 8, 2023)

Goals

- [] forecast The issues created as the followup to the design document are refined:
 -] [<https://gitlab.com/gitlab-org/gitlab/-/issues/426285>
 -] [<https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/issues/479>
 -] [<https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/issues/480>

Milestone 16.8 (December 11, 2023 - January 12, 2024)

Goals

- The effort on this key project are going to be pause due to limited backend availability and other priorities

Milestone 16.9 (January 15, 2024 - February 9, 2024)

Goals

- We should be able to resume the work on this project in this milestone, the expected MVC is due in 2 to 3 milestones.

Milestone 16.10 (February 12, 2024 - March 8, 2024)

Goals

-

Milestone 16.11 (March 11, 2024 - April 12, 2024)

Goals

-

Milestone 17.0 (April 15, 2024 - May 10, 2024)

Goals

-

SECTION: engineering/devops/ops/project-plans/container-registry-v2.md

Weekly Project Plan

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/5521>

Milestone 16.3 (June 18, 2023 - July 17, 2023)

Goals

- Building instructions for manual install of the metadata database
- Planning discussions on how to breakdown the project into Experimental and Beta phases
- Milestone plan: <https://gitlab.com/gitlab-org/gitlab/-/issues/416110> container-registry

Week of Aug 14-18

- Completed:
 - Updated the architecture blueprint with a new table structured for our feature based rollout approach: <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/128824>
 - Completed the POC for installing the database using Helm and Omnibus for new installations. Demo videos are posted in the issue for both installation paths.
 - Implemented Headway automation. All issues and MRs marked with the ~headway-self-managed-registry-db label will be added to tables in the epic.
 - Product is researching how GitLab has structured communication with users in other beta programs through the use of separate issues for enrollment and feedback. This also provides a communication channel for users who reach out to us through our Support counterpart.

Milestone 16.4 (August 18, 2023 - September 17, 2023)

Goals

- Continue to develop walkthroughs and demos for manual database installation <https://gitlab.com/gitlab-org/container-registry/-/issues/1068>
- Evaluate options for installing the database through Omnibus and Helm Charts <https://gitlab.com/gitlab-org/container-registry/-/issues/1102>
- Weekly forecasts for this project will be determined over the course of this milestone. This project is just getting started and will initially be reactive to research findings and user feedback.

Week of Aug 21-26

- Plan:
 - Define the features and draft changes for the feature based rollout approach discussed last week. This matrix of features will be documented in the architecture blueprint.
 - Officially opening the Beta program this week.
 - Continue testing data migration strategies with different types of storage backend and registry sizes.
 - Starting work on Omnibus and Helm Chart installations following discovery in the POC completed last week.
 - Experiment with enabling and configuring the metadata database in Omnibus installations. <https://gitlab.com/gitlab-org/container-registry/-/issues/1071>
- Completed:
 - Added information about the beta program to docs (MR in progress) including the list of features that will be used as iterations to progress beyond Beta.
 - Persist container registry database settings in Omnibus after a reconfiguration. <https://gitlab.com/gitlab-org/container-registry/-/issues/1071>

Week of Aug 28-Sept 1

- Plan:
 - Create issues for interaction and feedback with users. Identifying a first user to start working with the beta this week.
 - Discussions and research to provision the container registry database in Omnibus. <https://gitlab.com/gitlab-org/container-registry/-/issues/1102>
 - Testing different sizes of the registry for scalability.
 - Coordinate with our Support Liason to prepare for user interaction.
- Completed:
 - Created issues for communication with users regarding beta interest and feedback. Discussions are in progress with a potential first user for the beta program.
 - Through testing, the team discovered a problem with persisting database configuration which will need to be fixed in 16.4 before the beta user can start the installation.

Week of Sept 4-8

- Plan:
 - Review and merge the omnibus MR for persisting database settings.
 - Create walkthrough demo of the migration process (issue)
 - Add to the container registry seeder project to make testing easier.
- Completed:
 - Merged omnibus MR and tested.
 - Recorded youtube video for import process.
 - Tested 100GB model for migration in GCP. (issue)

Week of Sept 11-15

- Plan:
 - Investigate using a lock file for data integrity (issue). Prevent non-database registry changes (such as offline garbage collection) to database managed objects.
 - Further testing with AWS and 500GB models.

- Completed:
 - Made progress on technical design for lockfiles to protect data integrity in the metadata database.

Milestone 16.5 (September 17, 2023 - October 10, 2023)

Goals

- Open the Beta program for the container registry and start getting customer feedback about the docs and migration.
- Provision a Container Registry database for Omnibus installations (issue)
- Add observability metric in service ping to identify use of the new database in self-managed installs.
- Test offline migration runtime with test registries of different sizes and storage backends

Week of Sept 18-22

- Plan:
 - With the 16.4 release, the fix for persisting database settings will be available to self-managed and users can begin testing for the beta program.
 - Define detailed project delivery goals
- Completed:
 - Discussion with Geo on feasibility of data migrations and registry changes across geo sites.
 - Initial work on lockfiles
 - Add observability metric to service ping for container registry project.

Week of Sept 25-29

- Plan:
 - Engage with beta users and gather feedback from first installations.
 - Follow-up on discussion regarding geo support
 - Continue lockfiles implementation
- Completed:
 - Progress adding a database subcommand to Omnibus gitlab-ctl <https://gitlab.com/gitlab-org/container-registry/-/issues/1102>
 - Updated the quick start guide to prevent omnibus users from editing the generated registry config directly.
 - Finished Rails implementation of observability metric.

Week of Oct 2-6

- Plan:
 - Create a dashboard and review data for service ping metric.
 - Scaling back on lockfiles to only be used for disabling offline GC.
 - Continue on Omnibus gitlab-ctl subcommand.
- Completed:
 - Continued progress on Omnibus gitlab-ctl subcommand.
 - Completed observability metric work and created a dashboard for tracking db usage.

Week of Oct 9-13

- Plan:
 - Refine issues for addressing first phase of beta feedback
 - Research methods for testing data migration in an environment with Geo
- Completed:
 - Added support for a layer media type a user had trouble importing (issue)
 - Prevent offline gc from running if the database is in use (issue)

Milestone 16.6 (October 17, 2023 - November 10, 2023)

Goals

- Implement issues related to feedback from first phase of beta users
- Add registry migration subcommand to Omnibus gitlab-ctl
- Create import guides specific to omnibus

Week of Oct 16-20

- Plan:
 - Prepare container registry release for 16.5.
 - Respond to user feedback regarding errors for unknown layer media types.
- Completed:
 - Add registry database migrate command in omnibus is working in test environment
 - Additional testing in migration of large registries

Week of Oct 23-27

- Plan:
 - Deprecating OSS and Swift storage drivers: issue
 - Store Unknown Layer Blobs as Generic Media Types Rather than failing: issue
 - Adding ability to target a primary databases node when running post-migration issue
- Completed:
 - Import unknown media types as generic on parity with the API: issue
 - OSS and Swift are deprecated issue
 - Omnibus registry database migrate subcommand in review with Distribution MR
 - Wrapped up current scope of performance tests

Week of Oct 30-Nov 3 (Q3 ends on Oct 31)

- Plan:
 - Several interactions with users in beta issue to address.
 - Continue progress on Omnibus registry database migrate command MR.
- Completed:
 - Progress on the MR to add gitlab-ctl commands for the registry database MR

Week of Nov 6-10

- Plan:
 - Investigation into validation of self-managed imports epic

- Continue progress on Omnibus registry database migrate command MR
- Completed:
 - Omnibus registry database migrate command MR is in review

Week of Nov 13-17

- Plan:
 - Release issues related to feedback from first phase of beta users.
 - Build guides for omnibus installs including import and migrate commands.
 - Started a blog post to announce availability of the container registry database.
 - Developing plans to migrate .com only features (currently behind feature flags) to self-managed.
- Completed:
 - Evaluate adding a progress bar and better log presentation for the importer.
 - Engage with users and customers who are running the importer on large registries.
 - Created list of feature flagged container registry endpoints that exist in GitLab Rails for .com.

Milestone 16.7 (November 13, 2023 - December 21, 2023)

Goals

- Engage with beta users after 16.6 release improvements and gather feedback from self-managed installations.
- Create imported image validation command issue

Week of Nov 20-24

- Plan:
 - Finalize the MR for Omnibus registry database migrate command.
 - Completed:
 - Engage with users and customers who are running the importer on large repositories.
- Responding to feedback.

Week of Nov 27-Dec 1

- Plan:
 - Finalize progress bar and UX improvements for the importer.
 - Continue communication with beta customers who are testing the import process.
- Completed:
 - Discussion regarding expansion of usage ping metrics.
 - Draft of importer progress bar is ready for review.

Week of Dec 4-8

- Plan:
 - Evaluate options for validating images after they are imported.
 - Respond to feedback in Omnibus MR to add the import command.
- Completed:
 - Blog post published GitLabs next-generation container registry is now available

- Omnibus import command completed and merged.
- Added a comment at the completion of /offline/ garbage collection linking to the beta issue.
- Progress bar completed.

Week of Dec 11-15

- Plan:
 - Create plans for adding/testing new features for self-managed, now available to users with the metadata database.
 - Continue research into container image validation after import.
- Completed:
 - Evaluate tools for validating images after import. (issue)

Milestone 16.8 (December 22, 2023 - January 18, 2024)

Goals

- Implement issues related to feedback from second phase of beta users
- Validate Self-Managed Imports
- Utility support for registry database commands, such as in gitlab-ctl, equivalent to what is already present for offline garbage collection for charts

Week of Dec 18-22

- Plan:
 - Working with major registry user who will do their data migration over the holiday break.
- Most of the container registry team is unavailable this week.

Week of Dec 25-29

- Plan:
 - Continue working with major registry user on setup and migration of their registry database. Most of the container registry team is unavailable this week.
- Completed:
 - Migration for this user was a huge success! We'll address the follow-up issues in our next milestone.

Week of Jan 1-5

- Plan:
 - Catching up after holidays and continued engagement with beta users.
- Completed:
 - Reviewed customer migration successes and concerns, and developed follow-up issues.

Week of Jan 8-12

- Plan:
 - Release issues related to feedback from second phase of beta users.
 - Develop next steps plan and a communication campaign to increase visibility.

- Completed:
 - Reorganized the project plan and delivery targets issue.

Milestone 16.9 (January 13, 2024 - February 15, 2024)

Goals

- Create import guides specific to charts
- Test: Ensure container registry doesn't lose Geo support

Week of Jan 22-26

- Plan:
 - Start testing Geo support
 - Continue testing and documentation for manual Charts migration
- Completed:
 - Progress on Charts documentation issue
 - Merged Omnibus documentation docs

Week of Jan 29-Feb 2

- Plan:
 - Continue testing Geo support
 - Continue Charts migration documentation

Milestone 16.10 (February 16, 2024 - March 21, 2024)

Goals

- Automatically apply DB migrations during upgrades with Charts. (blocked)
- Importer: Retry Entire Step on Transient Error
- Importer: Pass on ErrDigestUnsupported
- Importer: Disable the Blob Descriptor Cache
- Importer: Reconsider the Clean Database Option

SECTION: engineering/devops/ops/project-plans/external-ci-keyword.md

Weekly Project Plan

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/10866>

Milestone 16.5 (September 17, 2023 - October 13, 2023)

Goals

- [-] [Agree on the implementation plan with the Verify Team]
- x] [Prepare a POC of the functionality]

Week of Oct 09-13

-] forecast: The [POC of the functionality is ready to be demoed and reviewed by the Verify team
- x] : The [POC of the functionality is ready and in review of the Verify team

Milestone 16.6 (October 17, 2023 - November 10, 2023)

Goals

- [] Agree on the implementation plan with the Verify Team
-] [Add CI job state for waiting for external action Is completed.

Week of Oct 16-20

- x] forecast: Progress on [Add CI job state for waiting for external action this is a complex issue that will require several weeks.

Week of Oct 23-27

- x] forecast: Progress on [Add CI job state for waiting for external action

Week of Oct 30 Nov 03

- x] forecast: [Add CI job state for waiting for external action is completed

Week of Nov 06-10

- Due to other concurring priorities we do not expect further work beside documenting the current state of art for the moment.

Milestone 16.7 (November 13, 2023 - December 8, 2023)

Goals

- We do not foresee any ability for the team to work on this effort in this milestone due to concurrent priorities

Milestone 16.8 (December 11, 2023 - January 12, 2024)

Goals

- We do not foresee any ability for the team to work on this effort in this milestone due to concurrent priorities

Milestone 16.9 (January 15, 2024 - February 9, 2024)

Goals

- We do not foresee any ability for the team to work on this effort in this milestone due to

concurrent priorities

SECTION: engineering/devops/ops/project-plans/gcp-secrets-integration.md

This project has been released.

Archive

<details><summary>Click to view past plans</summary>

Milestone 16.7 (November 13, 2023 - December 8, 2023)

Milestone Goals

- Complete the OIDC configuration for getting secrets from GCP
- Begin work on runner authentication with GCP

Week of November 20, 2023 (Milestone 16.7)

Team Capacity

- 1 BE

Goals

- <https://gitlab.com/gitlab-org/gitlab/-/issues/428402> Create POC and proposed solution for the integration with GCP Secrets Manager using GCP Workload Identity Pool.

Week of November 27, 2023 (Milestone 16.7)

Team Capacity

- 1 BE

Goals

- <https://gitlab.com/gitlab-org/gitlab/-/issues/428402> Finalize the proposed solution. We would have a clear understanding of what changes are needed on Rails, what information would be sent by Rails to Runner in the job payload, how Runner would use this job payload to authenticate to GCP and access the secrets.

Week of December 4, 2023 (Milestone 16.7)

Team Capacity

- 1 BE

Goals

- <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/138151> Merge the architecture blueprint MR.
- <https://gitlab.com/gitlab-org/gitlab/-/issues/428400> Begin development on runner authentication with GCP

Week of December 11, 2023 (Milestone 16.7)

Team Capacity

- 1 BE

Goals

- <https://gitlab.com/gitlab-org/gitlab/-/issues/428400> Continue development for runner authentication with GCP
- <https://gitlab.com/gitlab-org/gitlab/-/issues/428401> Begin development for rails support

Milestone 16.8 (December 18, 2023 - January 12, 2024)

Milestone Goals

- <https://gitlab.com/gitlab-org/gitlab/-/issues/428400> Complete runner authentication with GCP
- <https://gitlab.com/gitlab-org/gitlab/-/issues/428401> Complete development for rails support

</details

SECTION: engineering/devops/ops/project-plans/k8s-dashboard.md

Weekly Project Plan

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/11112>

Milestone 16.5 (September 17, 2023 - October 13, 2023)

Goals

- -] [Implement Watch API for Kubernetes API calls as this is a prerequisite for the dashboard
- -] [Design the dashboard

Week of Oct 09-13

- x] forecast: [Implement Watch API for Kubernetes API calls this issue is finalised or close to be, this represent a necessary step to create the dashboard

Milestone 16.6 (October 17, 2023 - November 10, 2023)

Goals

- x] [Implement Watch API for Kubernetes API calls as this is a prerequisite for the dashboard
- x] [Design the dashboard

Week of Oct 16-20

- -] forecast: [Implement Watch API for Kubernetes API calls is merged and tested
- [x] forecast: More issues are created to break down the work for the dashboard described in the epic

Week of Oct 23-27

- x] forecast: [Implement Watch API for Kubernetes API calls is merged and tested (behind feature flag)

Week of Oct 30 Nov 03

- x] forecast: [Implement Watch API for Kubernetes API calls feature flag is rolled out
- -] Now that the Iteration 1 of the dashboard is designed we can focus on refining [\[https://gitlab.com/groups/gitlab-org/-/epics/11351\]](https://gitlab.com/groups/gitlab-org/-/epics/11351)

Week of Nov 06-10

- Due to the limited availability of the main DRI for this week we do not expect much progress.
- A Conference Talk about the k8s dashboard has been delivered by Anna Vovchenko

Milestone 16.7 (November 13, 2023 - December 15, 2023)

Goals

- x] Now that the Iteration 1 of the dashboard is designed we can focus on refining [\[https://gitlab.com/groups/gitlab-org/-/epics/11351\]](https://gitlab.com/groups/gitlab-org/-/epics/11351)
- [x] 50% of the MVC issues for iteration 1 is delivered

Week of Nov 13-17

- [x] 40% of the frontend issues for Iteration 1 are refined
- x] Kubernetes Watch API is [enabled globally on gitlab.com
- x] Kubernetes Watch API is [enabled by default

Week of Nov 20-24

- [x] forecast: 60% of the frontend issues for Iteration 1 are refined
- x] forecast: MR to introduce [layout for dashboard pages is merged and tested
- x] forecast: MR to introduce [stats component for dashboard pages is merged and tested
- x] forecast: [Implement Watch API for Kubernetes services is merged and tested

Week of Nov 27 - Dec 1

- [x] forecast: 80% of the frontend issues for Iteration 1 are refined

- [x] forecast: MR to introduce table component for dashboard pages is merged and tested
- [x] forecast: MR to implement drawer component for dashboard pages is merged and tested
- -] forecast: [Iteration 1 of the Kubernetes pods page is done

Week of Dec 4-8

- [x] forecast: All frontend issues for Iteration 1 are refined
- x] forecast: [Iteration 1 of the Kubernetes pods page is done
- x] forecast: [Iteration 1 of the Kubernetes deployments page is done
- -] forecast: [Iteration 1 of the Kubernetes statefulSets page is done

Week of Dec 11-15

- x] forecast: [Iteration 1 of the Kubernetes statefulSets page is done
- x] forecast: [Iteration 1 of the Kubernetes replicaSets page is done

Milestone 16.8 (December 18, 2023 - January 12, 2024)

Goals

- [] Iteration 1 is finalised and deployed behind feature flags.
- [] Iteration 2 is scoped, the issues created and refined

Week of Dec 18-22

- x] forecast: [Iteration 1 of the Kubernetes daemonSets page is done
- x] forecast: [Iteration 1 of the Kubernetes jobs page is done

Week of Dec 25-28

- x] forecast: [Iteration 1 of the Kubernetes cronJobs page is done

Week of Jan 1-5

- Due to the limited availability of the main DRI for this weeeek we do not expect much progress.

Week of Jan 8-12

- x] forecast: [Iteration 1 of the Kubernetes services pag